

AI Healthcare Assistant using Hugging Face & Prompt Engineering

1. Introduction

This project focuses on developing an AI healthcare assistant using Hugging Face Transformers and prompt engineering techniques. The objective is to demonstrate how an instruction-tuned language model can be used to respond to healthcare-related questions and how carefully designed prompts can improve the structure and relevance of generated responses.

The project includes an interactive healthcare chatbot and several prompt engineering techniques such as role prompting, few-shot prompting, chain-of-thought prompt design, structured JSON output, and prompt optimization for reducing hallucinations.

The system is developed as an educational demonstration. It is not intended to provide definitive medical diagnoses or replace professional medical advice.

2. Model Selection

For this project, the `Qwen/Qwen2.5-0.5B-Instruct` model was selected from Hugging Face. It is an instruction-tuned language model and was used for generating responses to healthcare-related prompts.

The model was selected because its relatively small size makes it suitable for experimentation with limited computational resources. It was used with the Hugging Face Transformers library along with `AutoTokenizer`, `AutoModelForCausalLM`, and the text-generation pipeline.

A comparison was also made with the larger `Qwen/Qwen2.5-1.5B-Instruct` model. The larger model provides greater model capacity but requires more computational resources. The comparison helped demonstrate the trade-off between model size, resource requirements, and expected capabilities.

Although the selected 0.5B model was lightweight and easy to experiment with, testing showed that it could sometimes generate unsupported information or fail to follow strict formatting instructions. This limitation was considered while evaluating the prompt engineering techniques.

3. Prompt Engineering Techniques

Several prompt engineering techniques were implemented to improve the quality, structure, and relevance of the healthcare assistant's responses.

3.1 Role Prompting

Role prompting was used to assign a specific role to the model, such as a medical information assistant. In the bonus task, the same model was also adapted for Doctor, Nurse, Medical Student, and Patient roles.

This helped define the intended audience and communication style of the response.

3.2 Context and Constraints

Context was added to describe the user's situation, such as identifying the user as a beginner seeking general educational information.

Constraints were used to guide the model's behavior. For example, the prompts instructed the model not to provide definitive diagnoses, guess medical information, or add unrelated information.

3.3 Few-Shot Prompting

Few-shot prompting was used by providing three input-output examples before the final clinical note.

The examples demonstrated how clinical notes should be converted into a structured summary containing symptoms and severity. Additional rules were added to prevent the model from inventing symptoms or severity.

3.4 Chain-of-Thought Prompt Design

A chain-of-thought style prompt was designed for a patient with fever, cough, and an oxygen saturation of 88%.

The prompt instructed the model to carefully analyze the available information before producing a final recommendation. The internal reasoning or chain of thought was not requested in the final response.

3.5 Structured JSON Output

A structured output prompt was created to make the model return information using four predefined fields:

- disease
- symptoms
- risk_level
- recommendation

The prompt also instructed the model to return only JSON and avoid additional explanations.

3.6 Prompt Optimization

Prompt optimization was used to reduce hallucinations. The improved prompt instructed the model to use only information explicitly provided, avoid guessing diseases, avoid definitive diagnoses, and acknowledge when there was insufficient information.

4. Implementation and Tasks

The healthcare assistant was implemented using the Hugging Face Transformers library and the Qwen2.5-0.5B-Instruct model.

Task 1 — Hugging Face Chatbot

An interactive text-generation chatbot was developed using the Transformers pipeline. Users can enter medical questions and receive generated responses from the model.

Task 2 — Prompt Engineering

The broad prompt "Tell me about diabetes." was improved by adding a specific role, user context, constraints, and a predefined Markdown output structure.

Task 3 — Few-Shot Prompting

Three examples were provided to demonstrate how clinical notes should be converted into structured summaries. The final clinical note was then processed using the demonstrated format.

Task 4 — Chain-of-Thought Prompt Design

A prompt was designed for a patient with fever, cough, and an oxygen saturation of 88%. The model was instructed to carefully analyze the information and provide a final risk assessment and recommendation without revealing internal reasoning.

Task 5 — Structured JSON

A JSON-based prompt was created with the required fields `disease`, `symptoms`, `risk_level`, and `recommendation`. The model was instructed to return information using this predefined structure.

Task 6 — Model Comparison

The Qwen2.5-0.5B-Instruct model was compared with the larger Qwen2.5-1.5B-Instruct model based on parameters, use cases, advantages, and limitations.

Task 7 — Prompt Optimization

A poor response containing unsupported possible diseases was analyzed. The prompt was refined using constraints such as avoiding guesses, using only provided information, and acknowledging insufficient information.

Bonus — Role-Based Assistant

The same model was adapted for four different roles: Doctor, Nurse, Medical Student, and Patient. Role-specific instructions were used to change the communication style according to the intended user.

5. Challenges Encountered

During the implementation and testing of the healthcare assistant, several challenges were observed.

5.1 Hallucination and Unsupported Information

The Qwen2.5-0.5B-Instruct model sometimes generated information that was not explicitly provided in the prompt. For example, while processing simple patient symptoms such as fever and cough, the model sometimes introduced additional symptoms or possible medical conditions.

This showed that a language model can generate plausible-sounding information even when sufficient evidence is not available.

5.2 Inconsistent Instruction Following

The model did not always follow strict prompt instructions. In some experiments, it generated additional explanations even when it was instructed to return only a specific format such as JSON.

Similarly, some role-based prompts resulted in additional information that was not present in the original patient information.

5.3 Structured Output Limitations

The JSON prompting experiment showed that prompt instructions alone could not always guarantee perfectly formatted JSON output. The model occasionally generated explanatory text before or around the requested JSON structure.

5.4 Limited Model Capacity

The selected 0.5B model was lightweight and suitable for experimentation, but its smaller size resulted in limitations in instruction following and response reliability compared with what may be expected from larger models.

5.5 Generation Configuration Warning

During text generation, Transformers displayed a warning indicating that both `max_new_tokens` and `max_length` were configured. The warning did not prevent the model from generating responses, but it highlighted the need to manage generation settings carefully when preparing the final implementation.

6. Prompt Refinement and Improvements

Prompt refinement was performed to improve the reliability and relevance of the model's responses.

6.1 Initial Prompt

The initial prompts were relatively broad and provided limited instructions to the model. This resulted in responses that sometimes contained additional information, unsupported assumptions, or content that was not requested.

For example, a simple medical prompt could result in the model suggesting multiple possible diseases even when only a single symptom was provided.

6.2 Refined Prompt

The prompts were improved by adding explicit instructions and constraints. The refined prompts instructed the model to:

- Use only information explicitly provided by the user.
- Avoid guessing or inventing medical conditions.
- Avoid providing definitive diagnoses.
- Acknowledge when the available information is insufficient.
- Follow a predefined output format.

- Avoid unrelated explanations and additional content.

6.3 Observed Improvements

After prompt refinement, the model showed improvements in several areas.

First, the responses became more structured because specific output formats were provided. For example, the JSON task defined the exact fields that the model should use.

Second, few-shot examples helped the model understand the expected format for converting clinical notes into structured summaries.

Third, explicit constraints reduced direct disease speculation in some experiments and encouraged the model to acknowledge insufficient information.

However, the improvements were not completely consistent. The Qwen2.5-0.5B-Instruct model sometimes continued to generate unsupported information or failed to follow strict formatting instructions. This shows that prompt engineering can improve model behavior but cannot guarantee complete factual reliability.

7. Conclusion

This project demonstrated the development of an AI healthcare assistant using Hugging Face Transformers and prompt engineering techniques.

An instruction-tuned Qwen2.5-0.5B-Instruct model was used to build an interactive chatbot and experiment with different prompting approaches. Role prompting, few-shot prompting, chain-of-thought prompt design, structured JSON output, and prompt optimization were implemented as part of the project.

The experiments demonstrated that carefully designed prompts can improve the structure, relevance, and clarity of language-model responses. Few-shot examples helped demonstrate the desired output format, while explicit constraints helped reduce unsupported information in some cases.

However, testing also showed limitations of using a small language model for healthcare-related applications. The model occasionally generated information that was not explicitly provided or failed to follow strict formatting instructions. Therefore, prompt engineering should be considered an important control mechanism, but it does not guarantee completely reliable medical responses.

The developed assistant is intended only as an educational demonstration and should not be used as a replacement for qualified medical professionals or for making medical

decisions.